

Notation de de Bruijn et recursion dans le compilateur Faust

Notation de de Bruijn et recursion dans le compilateur Faust

Resume

Cette note synthetise deux idees distinctes mais souvent melangees: d'une part, la notation de de Bruijn comme technique classique de representation sans noms des variables liees; d'autre part, son emploi dans Faust comme representation intermediaire des groupes de feedback produits par l'operateur $\sim$. Le point central est le suivant: Faust n'utilise pas les indices de de Bruijn comme un simple artifice theorique, mais comme une IR de propagation adaptee a des groupes recursifs multi-sortie, causalises par `delay1`, avant conversion vers une forme symbolique plus commode pour les passes ulterieures.

La these defendue ici est nuancee. L'usage de de Bruijn hors du lambda-calcul pur n'est pas, en soi, une invention propre a Faust: la litterature decrit des representations de Bruijn pour des lieux multiples, des familles de syntaxes mutuellement recursives, et des portees recursives. En revanche, l'adaptation precise a des groupes de feedback de signaux, avec projections de slots et etagement `boxes` -> `signaux de Bruijn` -> `signaux symboliques` -> FIR, est une construction caracteristique du compilateur Faust.

1. La notation de de Bruijn dans son usage classique

Dans l'article fondateur de 1972, Nicolaas Govert de Bruijn propose de remplacer les noms de variables liees par des entiers donnant la distance jusqu'au lieu correspondant. Le but est explicite: eviter les complications de substitution et d'alpha-conversion en representation mecanisable.

Sous cette forme classique:

```
lambda x. x          -> lambda. 1
lambda x. lambda y. x -> lambda. lambda. 2
lambda x. lambda y. y -> lambda. lambda. 1
```

Le principe semantique est simple:

- 1 designe le lieu le plus proche;
- 2 designe le lieu juste au-dessus;
- les references sont donc exprimees par profondeur, non par nom.

Cette representation a deux consequences connues:

1. l'equivalence alpha devient une egalite structurelle;
2. les operations de substitution et de "lifting" deviennent purement structurelles.

Dans la litterature plus recente, la representation de Bruijn ne se limite plus au lambda-calcul unaire. Keuchel et Jeuring montrent explicitement qu'une representation bien scopee en indices de de Bruijn peut aussi decrire des lieux multiples, des portees sequentielles et des portees recursives. J'en deduis que l'idee generale "de Bruijn au-dela du lambda pur" n'est pas specifique a Faust; ce qui est specifique est le type de structure que Faust choisit d'encoder ainsi.

2. Ce que Faust fait de cette notation

Dans Faust, la source utilisateur ne contient pas de noeuds `DEBRUIJNREC` ou `DEBRUIJNREF`. La recursion est ecrite au niveau du langage de boites, via l'operateur `~` ou, dans l'API box, via `boxRec`. La documentation officielle de Faust rappelle deux faits importants:

- la phase semantique traduit le programme vers des signaux par propagation symbolique;
- la recursion introduit automatiquement un delai d'un echantillon pour garantir la causalite.

L'IR interne utilise alors deux formes principales:

Forme	Role
<code>DEBRUIJNREC(body)</code>	lieur d'un groupe recursif
<code>DEBRUIJNREF(level)</code>	reference a un groupe recursif englobant

L'adaptation Faust differe du lambda-calcul standard sur un point decisif: le lieu ne porte pas une seule variable, mais un groupe de sorties. Une reference recursive n'est donc pas exploitable seule; elle est presque toujours combinee a une projection de slot:

```
proj(i, DEBRUIJNREF(1))
```

Le motif central du feedback causal est alors:

```
delay1(proj(i, DEBRUIJNREF(1)))
```

Autrement dit, Faust ne se sert pas des indices de de Bruijn pour nommer des variables de lambda-termes, mais pour reperer le groupe recursif courant a l'interieur d'un graphe de signaux multi-sortie. C'est, a mon sens, la specificite la plus interessante de cette IR.

3. Des boxes recursives a la forme de Bruijn dans propagate

3.1 Schema general

Au niveau des boites, $A \sim B$ construit une composition recursive. Si

- $A : Li \rightarrow Lo$
- $B : Ri \rightarrow Ro$

alors la composition est bien formee quand $Ri \leq Lo$ et $Ro \leq Li$.

Dans le port Rust, le branchement `FlatNodeKind::Rec(left, right)` de `crates/propagate/src/lib.rs` suit essentiellement ce schema:

```
10 = [ delay1(proj(i, DEBRUIJNREF(1))) ]   pour i = 0..Ri-1
11 = propagate(right, 10)
12 = propagate(left, 11 ++ lift(inputs))
group = DEBRUIJNREC(list(12))
```

```
sortie[i] =
  if aperture(12[i]) > 0 then proj(i, group)
  else 12[i]
```

Trois ingredients sont structurants:

- les placeholders de feedback `delay1(proj(i, DEBRUIJNREF(1)))`;
- le `lift` des references libres quand on entre dans une recursion imbriquee;
- le test d'`aperture`, qui decide si une branche est reellement recursive.

3.2 Exemple simple

Exemple minimal:

```
process = + ~ *(0.5);
```

Le compilateur fabrique d'abord une graine de feedback:

```
delay1(proj(0, DEBRUIJNREF(1)))
```

Puis le chemin de retour `*(0.5)` la transforme, et le corps principal `+` construit schématiquement:

```
body0 =  
  add(  
    delay1(proj(0, DEBRUIJNREF(1))) * 0.5,  
    input(0)  
  )
```

```
group = DEBRUIJNREC([body0])  
out0 = proj(0, group)
```

Le point important est que `DEBRUIJNREF(1)` ne veut pas dire "la variable `W` du groupe courant". Il veut dire "le groupe recursif le plus proche"; le choix du slot est reporte a `proj(0, ...)`.

3.3 Formes recursives imbriquées

Une forme representative est:

```
inner = + ~ *(0.5);  
process = inner ~ *(0.25);
```

Ici, la recursion definie par `inner` est elle-meme placee sous un nouveau lieu recursif. La consequence est classique en de Bruijn: toute reference libre venant de l'exterieur doit etre relevee d'un niveau pour ne pas etre capturee par le lieu interne.

Schématiquement, on obtient une structure du type:

```
DEBRUIJNREC(                                <- groupe externe  
  ...  
  DEBRUIJNREC(                              <- groupe interne  
    pair(  
      DEBRUIJNREF(1),                       <- pointe vers l'interne  
      DEBRUIJNREF(2)                       <- pointe vers l'externe  
    )  
  )  
  ...  
)
```

Le port Rust encode cela par `liftN(...)`:

```
liftN(DEBRUIJNREF(n), threshold) =  
  DEBRUIJNREF(n)   si n < threshold  
  DEBRUIJNREF(n+1) sinon
```

Cette regle tres courte cache en fait trois idees:

- `threshold` se lit comme "a partir de quel niveau la reference est encore libre relativement a la portee ou l'on est en train d'entrer";
- une reference de niveau strictement inferieur a `threshold` est deja liee a l'interieur du sous-arbre courant, donc il ne faut surtout pas la changer;

- une reference de niveau superieur ou egal a `threshold` reste libre par rapport a la nouvelle portee, donc il faut la decaler d'un cran pour qu'elle continue de pointer vers le meme lieu logique apres insertion du nouveau `DEBRUIJNREC`.

Autrement dit, `lift` ne "rend pas tout plus profond". Il releve seulement la partie libre du sous-arbre.

Vu comme algorithme structurel complet, `liftn` agit plutot ainsi:

```
liftn(node, threshold):
  si node = DEBRUIJNREF(level):
    retourner DEBRUIJNREF(level)      si level < threshold
    retourner DEBRUIJNREF(level + 1)  sinon

  si node = DEBRUIJNREC(body):
    retourner DEBRUIJNREC(liftn(body, threshold + 1))

  sinon:
    reconstruire node en appliquant liftn a chaque enfant
```

Le `threshold + 1` est le point cle. Descendre sous un binder recursif interne signifie qu'un niveau de plus devient localement lie. Le critere de "liberte" doit donc etre decale lui aussi.

3.3.1 Exemple minimal: reference libre versus reference deja liee

Considérons le sous-arbre suivant, juste avant d'entrer dans une nouvelle recursion:

```
pair(
  DEBRUIJNREF(1),
  DEBRUIJNREC(DEBRUIJNREF(1))
)
```

Si l'on applique `liftn(..., 1)`, on obtient:

```
pair(
  DEBRUIJNREF(2),
  DEBRUIJNREC(DEBRUIJNREF(1))
)
```

La difference entre les deux branches est fondamentale:

- dans la branche gauche, `DEBRUIJNREF(1)` est libre par rapport a la nouvelle portee, donc il doit devenir `DEBRUIJNREF(2)`;
- dans la branche droite, le `DEBRUIJNREF(1)` est deja capture par le `DEBRUIJNREC` local, donc il doit rester 1.

Cet exemple montre pourquoi `lift` ne doit pas etre pense comme "ajouter 1 a toutes les references". Ce serait faux: on casserait les references deja liees dans les sous-groupes internes.

3.3.2 Exemple intuitif dans le pipeline Faust

Supposons qu'une valeur venant d'une recursion exterieure contienne deja:

```
delay1(proj(0, DEBRUIJNREF(1)))
```

et qu'on reutilise cette valeur a l'interieur d'un nouveau `Rec`. Sans `lift`, le `DEBRUIJNREF(1)` serait relu comme "le nouveau groupe interne", alors qu'il voulait dire "le groupe externe deja en place".

Apres `liftn(..., 1)`, on obtient:

```
delay1(proj(0, DEBRUIJNREF(2)))
```

et le sens lexical est preserve:

- DEBRUIJNREF(1) designe désormais le groupe interne nouvellement cree;
- DEBRUIJNREF(2) continue de designer l'ancien groupe externe.

Quand un sous-arbre provenant d'une recursion exterieure entre dans une recursion interieure, un DEBRUIJNREF(1) libre devient donc DEBRUIJNREF(2). Sans ce decalage, la reference serait capturee a tort par le nouveau DEBRUIJNREF.

3.3.3 Pourquoi Faust doit lifter a deux endroits

Dans `propagate`, cette operation est appliquee a deux familles d'objets:

- les `inputs` injectes dans le corps `left` du `Rec`;
- les valeurs stockees dans `slot_env`.

Le second point est facile a manquer, mais il est essentiel. Une valeur issue d'une abstraction de boites, d'une definition locale ou d'une fermeture peut elle aussi contenir des DEBRUIJNREF venant d'une boucle plus exterieure. Si elle est reinjectee telle quelle dans une boucle plus interne, elle sera capturee silencieusement par le mauvais binder.

Le role profond de `lift` dans Faust n'est donc pas seulement de "faire marcher les boucles imbriquees". C'est plus precisement de maintenir l'invariant suivant:

l'introduction d'un nouveau groupe recursif ne doit jamais changer accidentellement le leur logique vise par une reference libre preexistante.

3.4 Recursion multi-sortie et recursion mutuelle

Dans Faust, la recursion mutuelle est un cas particulier de la recursion multi-sortie, pas un synonyme. Un exemple de regression deja present dans le corpus est:

```
import("stdfaust.lib");

feedback = si.bus(2) ~ ((*0.5), *(0.25));
process = feedback;

DEBRUIJNREC([
  body0 = delay1(proj(0, DEBRUIJNREF(1))) * 0.5,
  body1 = delay1(proj(1, DEBRUIJNREF(1))) * 0.25
])
```

Au sens strict, cet exemple est surtout un cas multi-sortie: chaque canal se reboucle principalement sur lui-meme. Mais du point de vue du compilateur, le cas vraiment mutuellement recursif n'introduit pas un nouveau mecanisme: il change seulement quelles projections apparaissent dans chaque corps. Par exemple :

```
import("stdfaust.lib");

process = si.bus(2) ~ ((*0.5), *(0.25)) : ro.cross(2));

qui se lower schematiquement en :

DEBRUIJNREC([
  body0 = delay1(proj(1, DEBRUIJNREF(1))) * 0.25,
  body1 = delay1(proj(0, DEBRUIJNREF(1))) * 0.5
])
```

Cette fois, les deux sorties dependent l'une de l'autre, et pas seulement de leur propre slot.

Le point cle est donc le suivant: dans Faust, la "recursion mutuelle" n'est pas un second dispositif a cote de la recursion simple; c'est la meme representation, mais avec un corps de groupe liste et des projections de slots.

4. Formes recursives degeneres: production, diagnostic, simplification

Une forme recursive est dite degeneratee quand le groupe recursif materiel existe encore, mais qu'une ou plusieurs de ses sorties ne dependent plus vraiment du feedback. Cela arrive quand certaines branches du chemin recursif:

- ignorent explicitement leur entree recursive;
- ou deviennent closes apres propagation et simplification.

Le corpus contient un cas representatif:

```
import("stdfaust.lib");

N = 8;
gain = hslider("gain", 0.5, 0.0, 0.99, 0.01);
process = si.bus(N) ~ (!, !, !, !, !, !, !, *(gain));
```

Ici, les canaux 0..6 jettent leur entree recursive via !; seul le canal 7 reste vraiment recursif. L'outil conceptuel utilise pour le voir est l'`aperture`, definie comme le niveau maximal de reference de Bruijn libre dans un sous-arbre:

```
aperture(DEBRUIJNREF(k))      = k
aperture(DEBRUIJNREC(body))   = aperture(body) - 1
aperture(autre noeud)        = max(aperture(enfants))
```

Interpretation:

- `aperture > 0` : la branche est encore ouverte sur le groupe recursif;
- `aperture <= 0` : la branche est close a cette frontiere.

Dans `propagate`, une sortie close n'est deja plus reemise comme `proj(i, group)`. Mais cela ne supprime pas necessairement toute la degenerescence structurelle du groupe. Dans la pipeline C++ classique, une passe plus agressive `inlineDegenerateRecursions()` peut compacter un groupe et ne garder que les corps vraiment recursifs. Cela cree un probleme subtil: l'indice logique d'une projection peut rester celui d'origine, alors que l'arite physique du groupe a diminue.

Exemple typique:

```
avant compactage : proj(7, SYMREC([b0, ..., b7]))
apres compactage  : proj(7, SYMREC([b7]))
```

Le groupe n'a plus qu'un corps physique, mais la projection vaut encore 7. Pour un backend qui manipule les slots recursifs comme un `Vec`, c'est une forme instable voire invalide.

Dans le fast-lane Rust actuel, la simplification adoptee est plus etroite: dans `signal_prepare`, toute projection vers un groupe symbolique unaire est canonicalisee vers `proj(0, group)`. L'enjeu n'est donc pas seulement esthetique. Simplifier les formes degeneres sert a:

- garder des indices de slots denses;
- stabiliser le typage et le lowering FIR;
- eviter les erreurs de type "projection index out of bounds".

5. Conversion de la forme de Bruijn vers la forme symbolique

5.1 Pourquoi convertir

La forme de Bruijn est excellente pendant la propagation:

- pas de generation de noms;
- portees correctes par construction;
- partage structurel maximal dans l'arene.

En revanche, elle est peu lisible et peu commode pour les passes ultérieures. Compter mentalement les profondeurs de lieux au milieu d'un DAG de signaux est acceptable pour **propagate**, beaucoup moins pour le typage, la préparation FIR, le lowering des groupes recursifs et le diagnostic.

La forme symbolique remplace donc:

Forme de Bruijn	Forme symbolique
DEBRUIJNREC(body)	SYMREC(var, body)
DEBRUIJNREF(level)	SYMREF(var)

5.2 Algorithme

L'algorithme de `de_bruijn_to_sym(...)`, partagé entre le C++ historique et le port Rust, est conceptuellement:

```
convert(node):
  si node = DEBRUIJNREC(body):
    var = fresh("W")
    body1 = substitute(body, level=1, replacement=SYMREF(var))
    body2 = convert(body1)
    retourner SYMREC(var, body2)

  si node = DEBRUIJNREF(level):
    erreur: la racine convertie etait ouverte

  sinon:
    reconstruire recursivement le noeud
```

Le point subtil est la substitution:

```
substitute(node, level, repl):
  si aperture(node) < level:
    retourner node
  si node = DEBRUIJNREF(level):
    retourner repl
  si node = DEBRUIJNREC(body):
    retourner DEBRUIJNREC(substitute(body, level + 1, repl))
  sinon:
    reconstruire
```

Quand on descend sous un DEBRUIJNREC imbrique, le niveau recherche devient `level + 1`, exactement comme pour `liftn`. C'est ce qui permet de convertir correctement des arbres imbriqués.

5.3 Exemple imbrique

Le test `crates/tlib/tests/recursive_trees.rs` encode le cas minimal:

```
DEBRUIJNREC(
  DEBRUIJNREC(
    pair(DEBRUIJNREF(1), DEBRUIJNREF(2))
  )
)
```

La conversion donne:

```
SYMREC(W0,
  SYMREC(W1,
```

```

    pair(SYMREF(W1), SYMREF(W0))
  )
)
```

On retrouve la logique lexicale attendue:

- DEBRUIJNREF(1) dans le groupe interne devient SYMREF(W1);
- DEBRUIJNREF(2) dans le meme groupe devient SYMREF(W0).

5.4 Interet pour les phases ulterieures

Dans le port Rust actuel, `prepare_signals_for_fir(...)` clone toute la foret de sortie, applique `de_bruijn_to_sym(...)` sur la liste complete, puis effectue la normalisation des groupes unaires, le typage reduit, et enfin la preparation du lowering FIR.

Le lowerer FIR attend ensuite des groupes de la forme:

```

SYMREC(var, body_list)
SYMREF(var)
```

et non plus des noeuds DEBRUIJNREC / DEBRUIJNREF. Ce choix apporte trois benefices pratiques:

- il rend explicite l'identite du groupe recursif manipule;
- il permet a `signal_fir` de decoder directement la liste des corps d'un groupe symbolique;
- il fixe une frontiere de pipeline nette: apres preparation, le backend n'a plus a raisonner en termes de profondeurs lexicales.

En bref, la conversion n'est pas un simple embellissement. C'est un changement de representation qui separe la logique de portee, utile pendant la propagation, de la logique de consommation backend, utile pendant le lowering.

6. Conclusion

La notation de de Bruijn remplit dans Faust un role plus concret que dans de nombreux exposes theoriques: elle sert de forme de travail pour construire des groupes de feedback corrects par construction, y compris en presence d'imbrication, de multi-sortie et de partage structurel.

Le point le plus important n'est pas "Faust utilise de Bruijn", ce qui serait trop general, mais plutot:

- Faust traite la recursion comme un lieu de groupe, non comme une simple variable;
- les references recursives sont adressees par profondeur puis par projection de slot;
- les formes degeneratees imposent une discipline de canonicalisation;
- la conversion finale vers SYMREC / SYMREF isole les passes aval de la complexite de portee.

Vu sous cet angle, la forme de Bruijn dans Faust est a la fois classique dans son principe et tres specialisee dans son usage compilateur.

References

Sources externes

- N. G. de Bruijn, "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem", 1972. <https://research.tue.nl/en/publications/lambda-calculus-notation-with-nameless-dummies-a-tool-for-automat-2/>
- Faust Documentation, "Using the box API", sections "Faust compiler structure" et "Defining recursive signals". <https://faustdoc.grame.fr/tutorials/box-api/>

References sur l'usage de de Bruijn hors du lambda-calcul pur

- Theorie des types / cadres logiques / Automath:

- Fairouz Kamareddine, Alejandro Rios, "Pure Type Systems with de Bruijn Indices", *The Computer Journal*, 45(2), 2002. <https://doi.org/10.1093/comjnl/45.2.187>
- J.H. Geuvers, R.P. Nederpelt, "N.G. de Bruijn's contribution to the formalization of mathematics", *Indagationes Mathematicae*, 24(4), 2013. <https://doi.org/10.1016/j.indag.2013.09.003>
- Lieurs multiples, portees sequentielles et portees recursives:
 - Steven Keuchel, Johan T. Jeuring, "Generic Conversions of Abstract Syntax Representations", WGP 2012. <https://ics-archive.science.uu.nl/research/techreps/repo/CS-2012/2012-009.pdf>
- Higher-order rewriting et metatermes:
 - Eduardo Bonelli, Delia Kesner, Alejandro Rios, "A de bruijn notation for higher-order rewriting", RTA 2000. https://doi.org/10.1007/10721975_5
 - Eduardo Bonelli, Delia Kesner, Alejandro Rios, "de Bruijn Indices for Metaterms", *Journal of Logic and Computation*, 15(6), 2005. <https://doi.org/10.1093/logcom/exi051>
- Logique du premier ordre avec quantificateurs:
 - Manuel Eberl Wehr, Daniel Kirst, "Material dialogues for first-order logic in constructive type theory: extended version", *Mathematical Structures in Computer Science*, 2024. <https://www.cambridge.org/core/journals/mathematical-structures-in-computer-science/article/material-dialogues-for-firstorder-logic-in-constructive-type-theory-extended-version/17E117C76725C980F4EAA68F76203C77>
- Calcul des processus / metatheorie mechanisee:
 - Roly Perera, James Cheney, "Proof-relevant pi-calculus: a constructive account of concurrency and causality", *Mathematical Structures in Computer Science*, 2017. <https://www.cambridge.org/core/journals/mathematical-structures-in-computer-science/article/proofrelevant-calculus-a-constructive-account-of-concurrency-and-causality/952DC4F0B460B604B3F9047FC41FE04A>

Sources internes au depot

- recursion-debruijn-lowering-en.md
- flatnode-rec-to-signals-en.md
- crates/propagate/src/lib.rs
- crates/tlib/src/recursion.rs
- crates/transform/src/signal_prepare.rs
- crates/transform/src/signal_fir/module.rs
- tests/corpus/rep_79_multi_output_recursion.dsp
- tests/corpus/rep_71_degenerate_unary_recursion.dsp